

**UNIVERSITY OF RWANDA
COLLEGE OF SCIENCE AND TECHNOLOGY
SCHOOL OF ICT
COMPUTER AND SOFTWARE ENGINEERING DEPARTMENT
NYARUGENGE CAMPUS**

MODULE CODE/TITLE: WEB DESIGN

PROJECT TECHNICAL REPORT

WEB TECHNOLOGY TECHNICAL ASSESSMENT 2026

eHealth Hub: A Web-Based Healthcare Management Platform for Rwanda

Prepared by: Fulgence NIYOMUKIZA

Registration Number: 225056492

Year of Study: Year 1

Date: 19 July 2026

Table of Contents

Chapter 1. Introduction.....	4
1.1. Historical Background of the Case Study.....	4
1.2. Problem Statement.....	4
1.3. Objectives of the Project.....	4
General Objective.....	4
Specific Objectives.....	4
1.4. Expected Outcomes.....	4
1.5. Project Rationale (Importance to the Community).....	5
Chapter 2. System Analysis and Design.....	6
2.1. System Analysis.....	6
2.1.1 Intended Users of the Project.....	6
2.1.2 Intended System Partners.....	6
2.2. System Design.....	6
1. High-Level Architecture.....	6
2. UI Design (Sketch of the UI Module) and Flowcharts.....	6
3. Database Design with ERD and Relationships.....	8
Chapter 3. Implementation.....	10
3.1. Introduction of the Section.....	10
3.2. Screenshots with Captions and Discussion.....	10
3.2.1. Homepage — Login and Patient Registration.....	10
3.2.2. Patient Dashboard.....	11
3.2.3. Book Appointment.....	11
3.2.4. Appointment Status.....	12
3.2.5. Medical Records.....	12
3.2.6. Patient Profile.....	13
3.2.7. Doctor Dashboard.....	13
3.2.8. Doctor Appointments.....	14
3.2.9. Patient Reports (Medical Report Authoring).....	14
3.2.10. Doctor Availability.....	15
3.2.11. Admin Dashboard.....	15
3.2.12. Manage Doctors.....	16
3.2.13. Assign Doctor to Appointment.....	16
3.2.14. Manage Patients.....	17
3.2.15. Appointment Management.....	17
Chapter 4. Conclusion and Recommendation.....	18
Recommendations.....	18
To the Lecturer, about Technicalities.....	18
To the School, about Time Constraints.....	18
To the Partners (Healthcare Institutions Across Rwanda).....	18
To the User, about Training.....	18
To the Beneficiaries (Patients and Students).....	18
Appendix.....	19

Table of Figures

Figure 1: High-level architecture of the eHealth Hub platform.....	6
Figure 2: Landing page UI sketch: login and patient registration.....	7
Figure 3: Flowchart of the patient appointment booking process.....	8
Figure 4: Entity Relationship Diagram of the eHealth Hub data model.....	9
Figure 5: Homepage showing role-based portal links, the login form, and the patient self-registration form	10
Figure 6: Patient dashboard with quick links and personal account summary.....	11
Figure 7: Appointment booking form with department selection and doctor listing.....	11
Figure 8: Appointment status page showing confirmed, pending, and completed counts.....	12
Figure 9: Patient medical records page with healthcare history and current treatments.....	12
Figure 10: Patient profile page for viewing and updating personal information.....	13
Figure 11: Doctor dashboard summarising assigned appointments and reports.....	13
Figure 12: Doctor's assigned-patient list with department, problem, and status.....	14
Figure 13: Doctor's medical report form for recording diagnosis and treatment.....	14
Figure 14: Doctor availability scheduling form and current weekly schedule.....	15
Figure 15: Administrator dashboard with system-wide statistics.....	15
Figure 16: Doctor registration and management page.....	16
Figure 17: Administrator interface for assigning a doctor to a pending appointment.....	16
Figure 18: Administrator view of registered patients and monitoring statistics.....	17
Figure 19: Administrator view of all hospital appointments and their status.....	17

List of Abbreviations

UI	— User Interface
UX	— User Experience
HTML	— HyperText Markup Language
CSS	— Cascading Style Sheets
JS	— JavaScript
ERD	— Entity Relationship Diagram
API	— Application Programming Interface
DB	— Database
ICT	— Information and Communication Technology
RSSB	— Rwanda Social Security Board
MoH	— Ministry of Health (Rwanda)

Chapter 1. Introduction

1.1. Historical Background of the Case Study

Most healthcare facilities I looked at while planning this project, especially the smaller ones, still run on paper. Patients queue at a reception desk, a nurse writes their name in a register, and if a doctor is needed, someone walks over to check who is free. Rwanda has pushed hard on digitizing public services over the last few years, but that push hasn't reached every district or sector health centre yet, and a lot of day-to-day coordination between a patient, a doctor, and hospital administration still happens by hand.

That gap is what eHealth Hub is trying to close, at least at prototype level. For this Web Design assessment, I chose to build a system that isn't tied to one specific hospital, but could realistically be reused by any clinic or health centre in the country, since booking an appointment, assigning a doctor, and writing up a diagnosis are things every facility does regardless of size or location.

1.2. Problem Statement

From talking to people and reading about how Rwandan health centres currently operate, four recurring problems stood out to me:

- Patients often wait a long time just to get an appointment with a doctor.
- Medical records are scattered across paper files, which makes them easy to lose or misplace.
- There's no quick way for a patient to communicate with hospital staff outside of showing up in person.
- Once a patient books an appointment, they have almost no way of tracking what happens to it next.

None of these problems are unique to one hospital — they show up wherever manual, paper-based processes are still the norm. eHealth Hub tries to fix this by putting booking, doctor assignment, and record access into one web application that anyone with a browser can use.

1.3. Objectives of the Project

General Objective

To design and build a web-based platform that digitizes patient registration, appointment booking, doctor assignment, and medical record management for healthcare facilities across Rwanda.

Specific Objectives

- Build a role-based login so Patients, Doctors, and Administrators each see only what's relevant to them.
- Let patients register, book appointments by department, and check their appointment status without calling anyone.
- Give administrators a way to manage doctor and patient records and assign doctors to appointments that are still pending.
- Let doctors see who they're assigned to, write up diagnosis/treatment reports, and set their own weekly availability.
- Keep the interface simple enough that someone could use it the first time without training.

1.4. Expected Outcomes

By the time the project was done, I wanted a prototype where:

- A patient could register, log in, book an appointment, and see its status — all without needing staff to step in.
- An administrator could see system-wide numbers at a glance and assign doctors to whoever is waiting.
- A doctor could check their patient list, submit a report, and update their availability.
- Everything a user does in one sitting is remembered by the browser, so the whole flow can be demonstrated start to finish.

1.5. Project Rationale (Importance to the Community)

I designed this to be generic on purpose, so it isn't only useful to one hospital in Kigali — it could work for a clinic in any province. For patients, that means less time standing in line and more visibility into what's happening with their appointment. For hospital staff, it means less time spent manually coordinating who sees which doctor. And because it only needs a browser and basic web hosting to run, it's realistic even for facilities that don't have a big IT budget, which matters a lot in a country where health centres vary widely in resources.

Chapter 2. System Analysis and Design

2.1. System Analysis

2.1.1 Intended Users of the Project

- Patients — register, book appointments, track status, and view their own records.
- Doctors — manage assigned appointments, write diagnosis/treatment reports, and set weekly availability.
- Administrators — manage doctor and patient accounts, assign doctors to pending appointments, and keep an eye on overall activity.

2.1.2 Intended System Partners

- Healthcare facilities anywhere in Rwanda that want a lightweight digital front desk without a big IT investment.
- Rwanda's Ministry of Health and related digital-health initiatives, as a possible partner for wider rollout.
- Health insurance schemes such as RSSB / Mutuelle de Santé, for future billing and coverage checks.
- Internet and mobile providers, since the whole platform runs through a browser rather than installed software.

2.2. System Design

1. High-Level Architecture

Right now the system is entirely client-side. The pages (HTML5/CSS3) are what the user sees, plain JavaScript handles validation and page logic, and the browser's localStorage acts as the data store for users, appointments, reports, and availability. That's enough to prove the concept works, but a real deployment would need an actual backend and database behind it — the diagram below shows both the current setup and where it would go next. It's drawn in the same box-and-arrow style you'd get from a tool like draw.io, since that's the clearest way to show layers talking to each other.

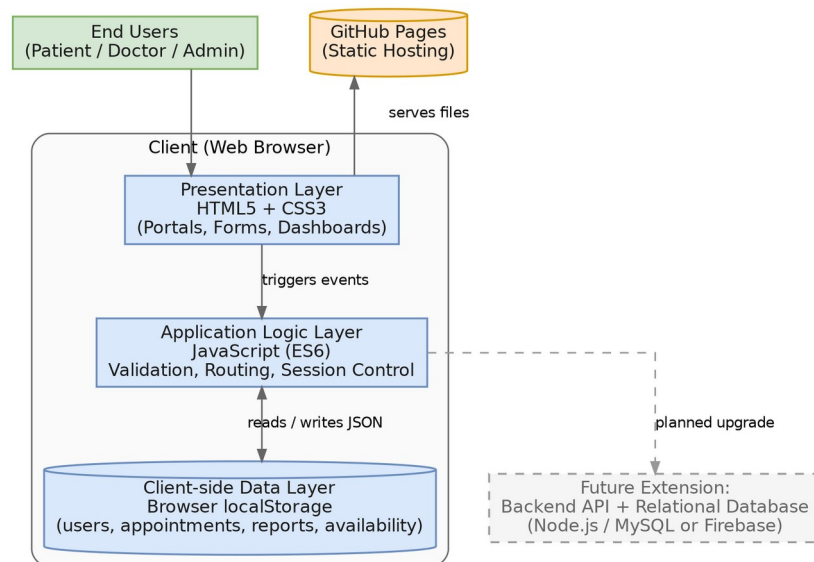


Figure 1: High-level architecture of the eHealth Hub platform

2. UI Design (Sketch of the UI Module) and Flowcharts

Everything starts from one landing page with three portal links — Patient, Doctor, Admin — plus the login and registration forms shown below.

eHealth Hub

Patient Portal | Doctor Portal | Admin Portal

Smart Healthcare Management Platform

The eHealth Hub is a digital healthcare management system designed to improve communication between hospitals, doctors, patients and administrators. The platform helps healthcare institutions manage appointments, patient records and healthcare services efficiently.

Healthcare Problems

- Long waiting time for appointments.
- Poor medical records management.
- Slow communication between patients and hospitals.
- Difficulty tracking appointments.

Digital Solutions

- Online appointment booking system.
- Electronic healthcare records.
- Fast communication system.
- Real time appointment updates.

System Benefits

- 24/7 healthcare access.
- Improved healthcare management.
- Better patient experience.
- Efficient hospital operations.

Emergency Services

Access emergency healthcare information and hospital communication services quickly.

Online Consultation

Patients can request appointments and communicate with doctors digitally.

Medical Records

Secure storage and easy access to patient healthcare information.

Welcome To eHealth Hub

Login or register to access digital healthcare services.

User Login

Email Address
Enter email address

Password
Enter password

User Role
Patient

Login

Patient Registration

Full Name
Enter full name

Email Address
Enter email

Phone Number
Enter phone number

Date Of Birth

Gender
Male

Password
Create password

Register

Healthcare Statistics

- 1,240 Registered Patients
- 58 Active Doctors
- 320 Monthly Appointments
- 15 Healthcare Departments

Emergency Contacts

Emergency Line: +250 788 000 000
Ambulance Service: +250 788 111 111
Email: support@ehealthhub.com

Figure 2: Landing page UI sketch: login and patient registration

Once logged in, a user lands on a dashboard specific to their role. The flowchart below traces a patient's appointment from the moment they log in to the moment a doctor closes it out, using standard flowchart shapes (rounded process boxes, a diamond for the validation check, and terminators for start/end).

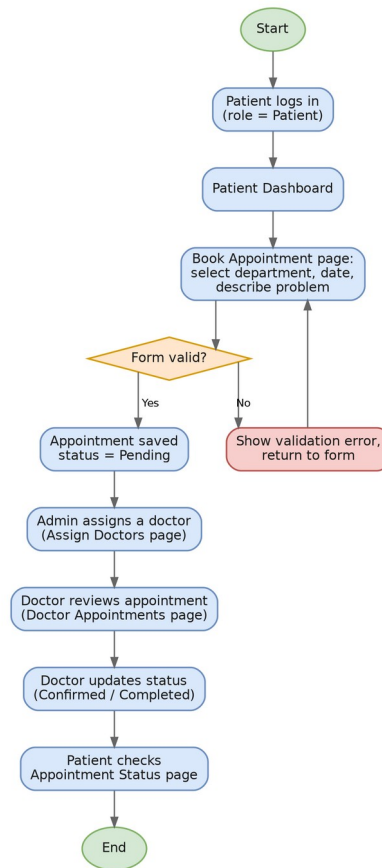


Figure 3: Flowchart of the patient appointment booking process

3. Database Design with ERD and Relationships

The prototype keeps its data in the browser instead of a real database, but the structure underneath is consistent enough to draw as an ERD. There are four entities: User (which covers Patients, Doctors, and Admins through a role field), Appointment, Medical Report, and Availability. A patient creates appointments; a doctor gets linked to appointments once assigned, plus their own availability slots and the reports they write. Each appointment can lead to at most one medical report once the visit wraps up. If this were rebuilt with a real database, these four entities would map almost directly onto relational tables.

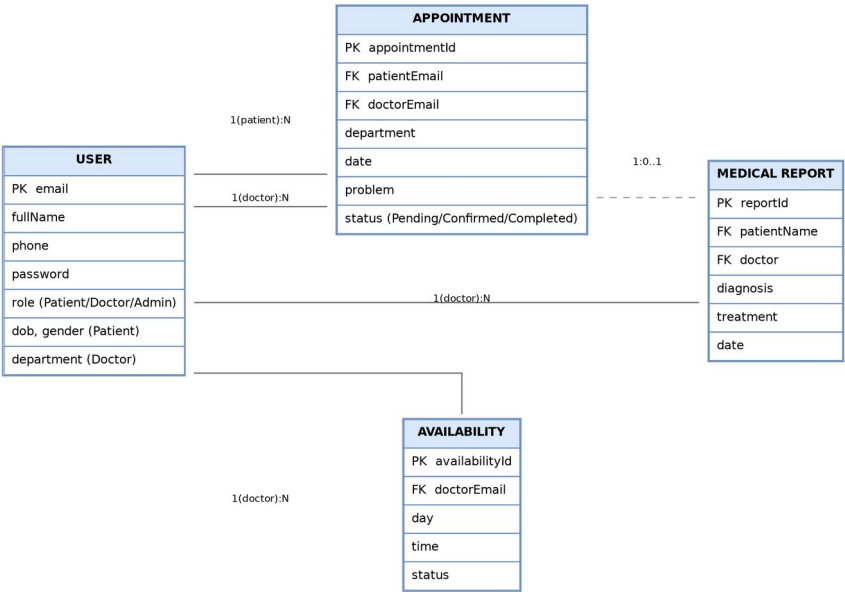


Figure 4: Entity Relationship Diagram of the eHealth Hub data model

Chapter 3. Implementation

3.1. Introduction of the Section

This chapter walks through every major page of the finished platform. It's built with plain HTML5, CSS3, and vanilla JavaScript (ES6), and it persists data through the browser's localStorage API rather than a server. The whole thing is hosted as a static site on GitHub Pages, with the source on GitHub (link in the Appendix). Each section below shows one screenshot and explains what it does and why I built it that way.

3.2. Screenshots with Captions and Discussion

3.2.1. Homepage — Login and Patient Registration

The screenshot displays the eHealth Hub homepage. On the left, a dark sidebar contains the 'eHealth Hub' logo and three role-based portal links: 'Patient Portal', 'Doctor Portal', and 'Admin Portal'. Below these links is the title 'Smart Healthcare Management Platform' followed by a brief description. A central image shows a doctor interacting with a patient. To the right of the image are three columns of text: 'Healthcare Problems' (Long waiting time, Poor medical records, Slow communication, Difficulty tracking appointments), 'Digital Solutions' (Online appointment booking system, Electronic healthcare records, Fast communication system, Real-time appointment updates), and 'System Benefits' (24/7 healthcare access, Improved healthcare management, Better patient experience, Efficient hospital operations). Below these columns are three more sections: 'Emergency Services' (Access emergency healthcare information and hospital communication services quickly), 'Online Consultation' (Patients can request appointments and communicate with doctors digitally), and 'Medical Records' (Secure storage and easy access to patient healthcare information).

On the right side of the page, there is a 'Welcome To eHealth Hub' section with a sub-header 'Login or register to access digital healthcare services.' Below this are two forms: 'User Login' and 'Patient Registration'. The 'User Login' form includes fields for 'Email Address', 'Password', and 'User Role' (a dropdown menu with 'Patient' selected), and a 'Login' button. The 'Patient Registration' form includes fields for 'Full Name', 'Email Address', 'Phone Number', 'Date Of Birth', 'Gender' (a dropdown menu with 'Male' selected), and 'Password', and a 'Register' button. Below the forms are two sections: 'Healthcare Statistics' (1,240 Registered Patients, 58 Active Doctors, 320 Monthly Appointments, 15 Healthcare Departments) and 'Emergency Contacts' (Emergency Line: +250 788 000 000, Ambulance Service: +250 788 111 111, Email: support@ehealthhub.com).

Figure 5: Homepage showing role-based portal links, the login form, and the patient self-registration form

This is the first thing anyone sees. Three portal links sit at the top (Patient, Doctor, Admin), followed by a quick overview of the problems the platform is trying to solve and the solutions it offers. On the right are the two forms that matter most: a shared login form where you pick your role along with your email and password, and a registration form for new patients (name, email, phone, date of birth, gender, password). The JavaScript behind these forms checks that nothing is left blank, that the email looks like an email, that the phone number is digits only, and that the password is at least six characters — all before anything gets saved. I also made the system auto-create a default admin account the first time it loads, so there's no separate setup step just to see the Admin Portal working.

3.2.2. Patient Dashboard

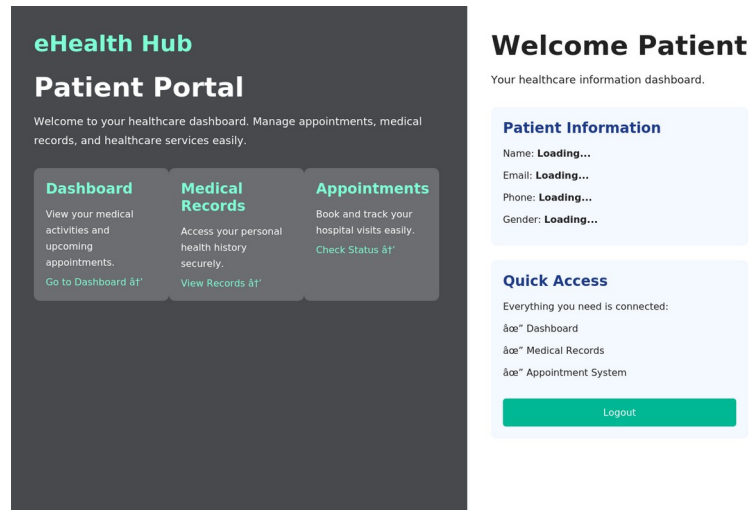


Figure 6: Patient dashboard with quick links and personal account summary

Once a patient logs in, this is home base. It greets them by name and gives three shortcuts — Dashboard, Medical Records, Appointments — each with a one-line description so it's obvious what each one does. Below that sits a Patient Information panel pulling the account's name, email, phone, and gender straight from the current session, plus a Quick Access summary and a logout button. I kept this page deliberately simple: its only job is to get the patient to whichever feature they actually came for.

3.2.3. Book Appointment

Figure 7: Appointment booking form with department selection and doctor listing

This is where a patient actually requests an appointment — department, preferred date, and a short description of the problem, with their name already filled in from the session. Submitting the form checks that every field is filled before it creates a new appointment with a status of "Pending." Underneath the form I listed a few available doctors with a short specialty blurb, mostly so the patient has some idea of who might end up handling their case once an admin makes the assignment. Since this form is where new demand enters the whole system, I made the validation here stricter than on most other pages.

3.2.4. Appointment Status

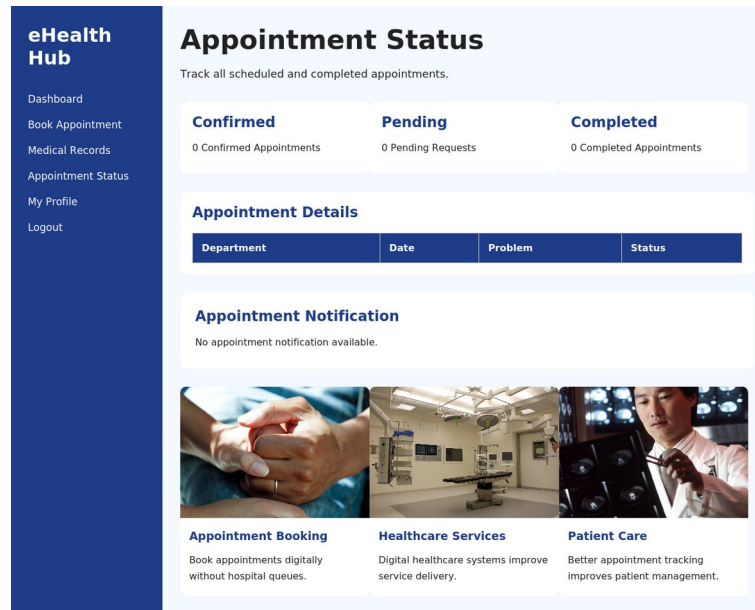


Figure 8: Appointment status page showing confirmed, pending, and completed counts

This page exists so a patient isn't left guessing what happened after they booked something. Three cards summarise how many of their appointments are Confirmed, Pending, or Completed, and a table below lists each one's department, date, problem, and status. The page only ever shows appointments tied to the logged-in patient's email — even in a prototype built on localStorage, I wanted to make sure one patient can't accidentally see another patient's bookings.

3.2.5. Medical Records



Figure 9: Patient medical records page with healthcare history and current treatments

Here a patient can look back at their own clinical history: a visit counter, their latest diagnosis, available prescriptions, and a Healthcare History table with date, doctor, diagnosis, and treatment. All of this is actually written by a doctor on the Patient Reports page and just filtered here to match the patient's name. That split — doctors write, patients only read — is intentional; it mirrors how real medical records work, where clinical staff control what gets entered.

3.2.6. Patient Profile

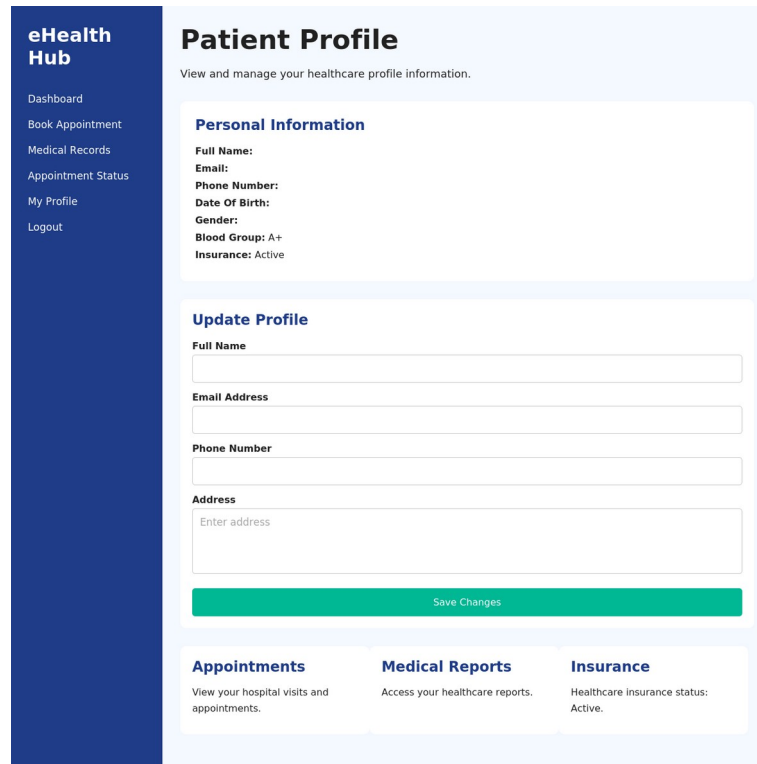


Figure 10: Patient profile page for viewing and updating personal information

A fairly standard account page: name, email, phone, date of birth, gender, plus a couple of illustrative fields like blood group and insurance status. Below that sits a form to update the name, email, phone, or address, along with shortcuts to Appointments and Medical Records. I added this mostly because every real portal needs some way to fix a typo in your own details without having to call an administrator.

3.2.7. Doctor Dashboard

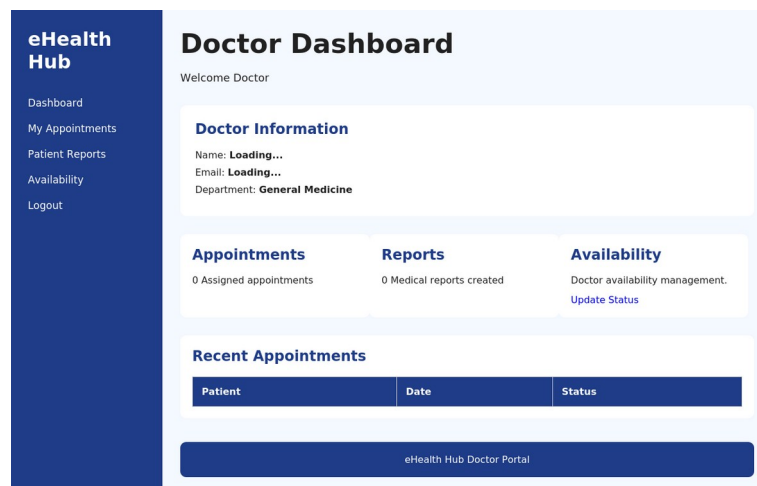


Figure 11: Doctor dashboard summarising assigned appointments and reports

This is what a doctor sees first — their name, email, and department, plus a running count of assigned appointments and reports they've already written. An Availability panel links to the schedule page, and a Recent Appointments table shows who they're currently attached to. My goal with this page was to answer one question immediately: who do I need to see, and what have I already documented for them?

3.2.8. Doctor Appointments



Figure 12: Doctor's assigned-patient list with department, problem, and status

This lists every appointment an admin has assigned to that doctor — patient name, date, department, problem, status, and an action to update that status (say, from Confirmed to Completed). Below the table I added the doctor's shift hours and a small emergency-case counter, mostly to give the page a sense of workload rather than just being a bare list.

3.2.9. Patient Reports (Medical Report Authoring)

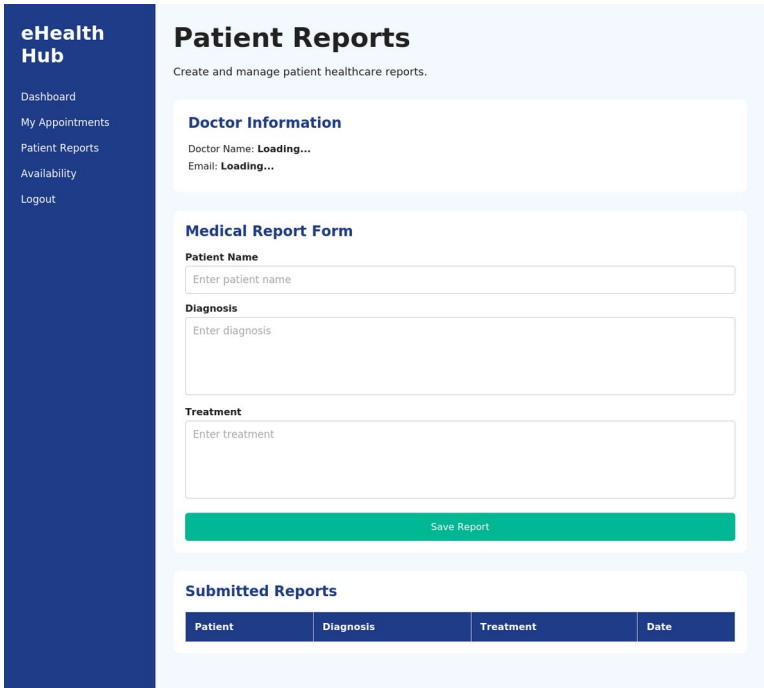


Figure 13: Doctor's medical report form for recording diagnosis and treatment

This is where a doctor writes up what happened during a visit — patient name, diagnosis, treatment. Saving it appends a new report to the shared reports store, tagged with the doctor's name and the date, and a table below lists everything that doctor has submitted so far. Functionally, this page is the other half of the Medical Records page patients see — one writes, the other reads.

3.2.10. Doctor Availability

Figure 14: Doctor availability scheduling form and current weekly schedule

A doctor uses this to say which days and hours they're free — pick a day, enter a time range, mark a status, and it gets saved against their email. A table below shows everything they've already submitted. The idea is that an administrator assigning doctors to appointments should be able to respect these hours instead of guessing.

3.2.11. Admin Dashboard

Figure 15: Administrator dashboard with system-wide statistics

This is the control centre for whoever logs in as an administrator. Cards at the top show total patients, active doctors, and pending appointments, each linking straight to the relevant management page. A System Information panel lists things like hospital name and department count, and a Recent Appointments table gives a live view of booking activity across the whole system, not just one doctor or patient.

3.2.12. Manage Doctors

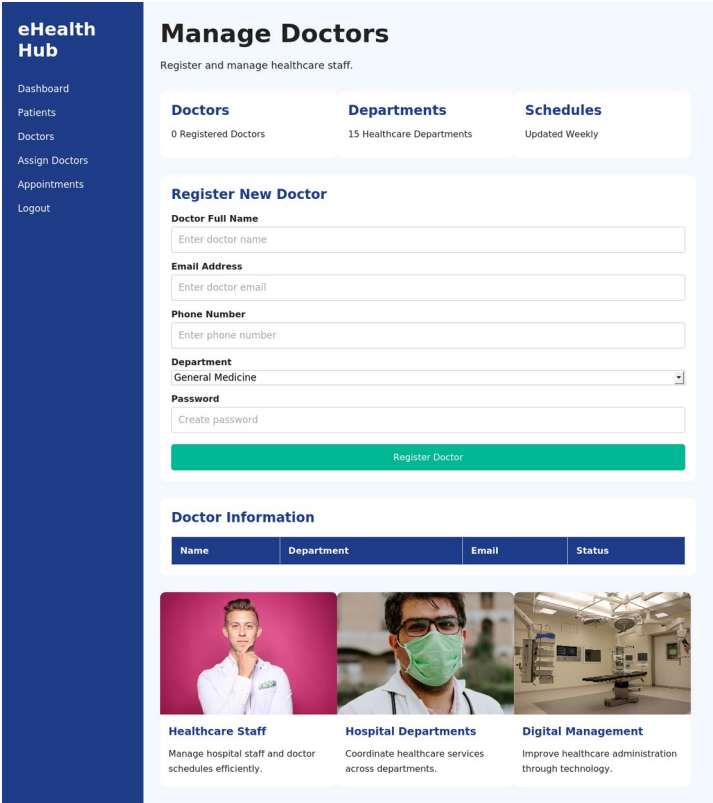


Figure 16: Doctor registration and management page

An administrator uses this to register new doctor accounts — name, email, phone, department, password — and to see every doctor currently in the system in one table. Cards above the form show doctor count, department count, and how often schedules get updated. Keeping doctor onboarding here, rather than letting doctors self-register, was a deliberate choice so the doctor list stays something an admin can actually vouch for.

3.2.13. Assign Doctor to Appointment

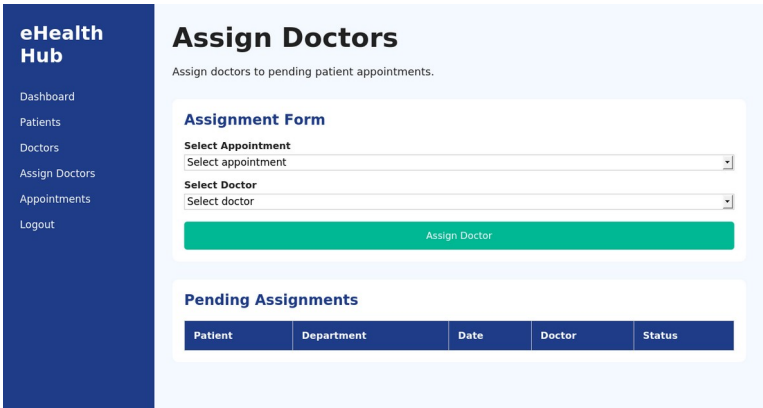


Figure 17: Administrator interface for assigning a doctor to a pending appointment

This page is arguably the most important one for the admin role, since it's where a booked appointment actually gets a doctor attached to it. Two dropdowns — one for pending appointments, one for doctors — plus a confirm button, and a table underneath tracking what's still waiting. Without this step, every appointment a patient books would just sit there unassigned.

3.2.14. Manage Patients

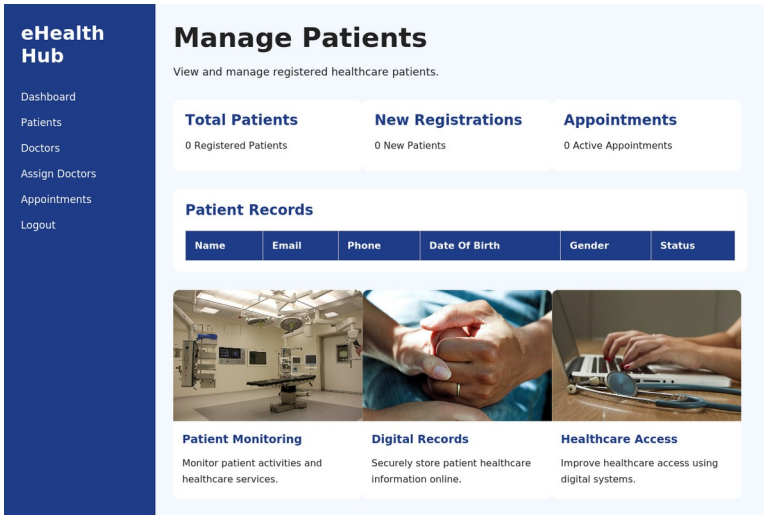


Figure 18: Administrator view of registered patients and monitoring statistics

Gives an administrator oversight of everyone registered as a patient — counts for total patients, new registrations, and active appointments, followed by a table of names, emails, phone numbers, dates of birth, gender, and status. It's really just the admin-side mirror of what a patient sees on their own profile, but for the whole population at once.

3.2.15. Appointment Management

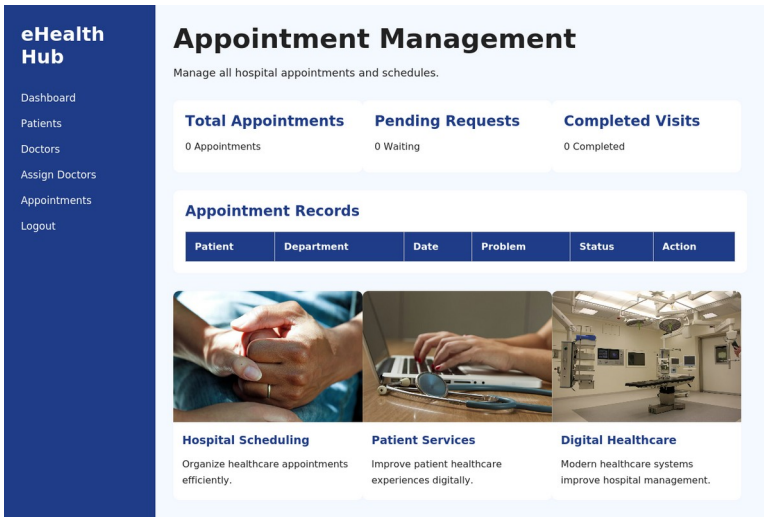


Figure 19: Administrator view of all hospital appointments and their status

The master list — every appointment in the system, not filtered by patient or doctor. Cards summarise totals, pending requests, and completed visits, and the table below lists each one's patient, department, date, problem, and status. I built this so that whoever runs the facility can spot bottlenecks, like a department accumulating too many pending requests, without digging through individual patient or doctor pages.

Chapter 4. Conclusion and Recommendation

Looking back at what I set out to do in Chapter 1, I think eHealth Hub gets there for what it was meant to be: a Web Design module project, not a production system. All three roles — Patient, Doctor, Admin — work end to end, and you can actually walk through a full story: a patient books an appointment, an admin assigns a doctor, the doctor writes up a report, and the patient sees it reflected in their own records. That loop closing was the main thing I wanted to prove was possible with just HTML, CSS, and JavaScript.

If I had to put a number on it, I'd say around 85–90% of the objectives from Section 1.3 are met at the prototype level. What's missing is mostly what you'd expect from a front-end-only build: there's no real server, nothing persists beyond the browser it was created in, and passwords are stored in plain text rather than hashed. None of that is a surprise for a Web Design assessment, but it's worth being upfront about, since anyone reading this shouldn't mistake it for something ready to run a real hospital.

Recommendations

To the Lecturer, about Technicalities

Since this module is really about front-end skills rather than backend engineering, I'd suggest grading criteria explicitly treat `localStorage` as an acceptable stand-in for a database at this stage — as long as the student can explain, like I've tried to in Section 2.2.1, what a real backend would look like on top of it.

To the School, about Time Constraints

Building three separate portals with distinct roles inside one assessment window is a lot to fit in. I'd recommend either giving this kind of multi-role project more time, or pairing it with a backend module so a database-backed version could follow without rushing the front-end work.

To the Partners (Healthcare Institutions Across Rwanda)

Any facility interested in something like this should treat it as a proof of concept, not something to deploy as-is. Before real use, it would need a proper backend, a secure database, hashed passwords, and a review against Rwanda's health-data protection rules.

To the User, about Training

The interface is meant to be self-explanatory, but a short walkthrough for receptionists, doctors, and admins — especially around the Assign Doctors and Availability pages — would cut down on input mistakes early on.

To the Beneficiaries (Patients and Students)

Patients relying on a system like this should keep their login details private and flag anything incorrect in their records quickly, since a future production version would only be as reliable as the data people put into it.

Appendix

Source code repository (GitHub):

<https://github.com/thepillar/eHealth-Hub>

Live deployed application (GitHub Pages):

<https://thepillar.github.io/eHealth-Hub/>

The repository is public on GitHub, so no account is needed to view the source code or the deployed site. The admin account seeded automatically on first load uses admin@ehealth.com / admin123, for demo and grading purposes only.